Module 4

IEEE 1800-2005 (SystemVerilog Design Subset)

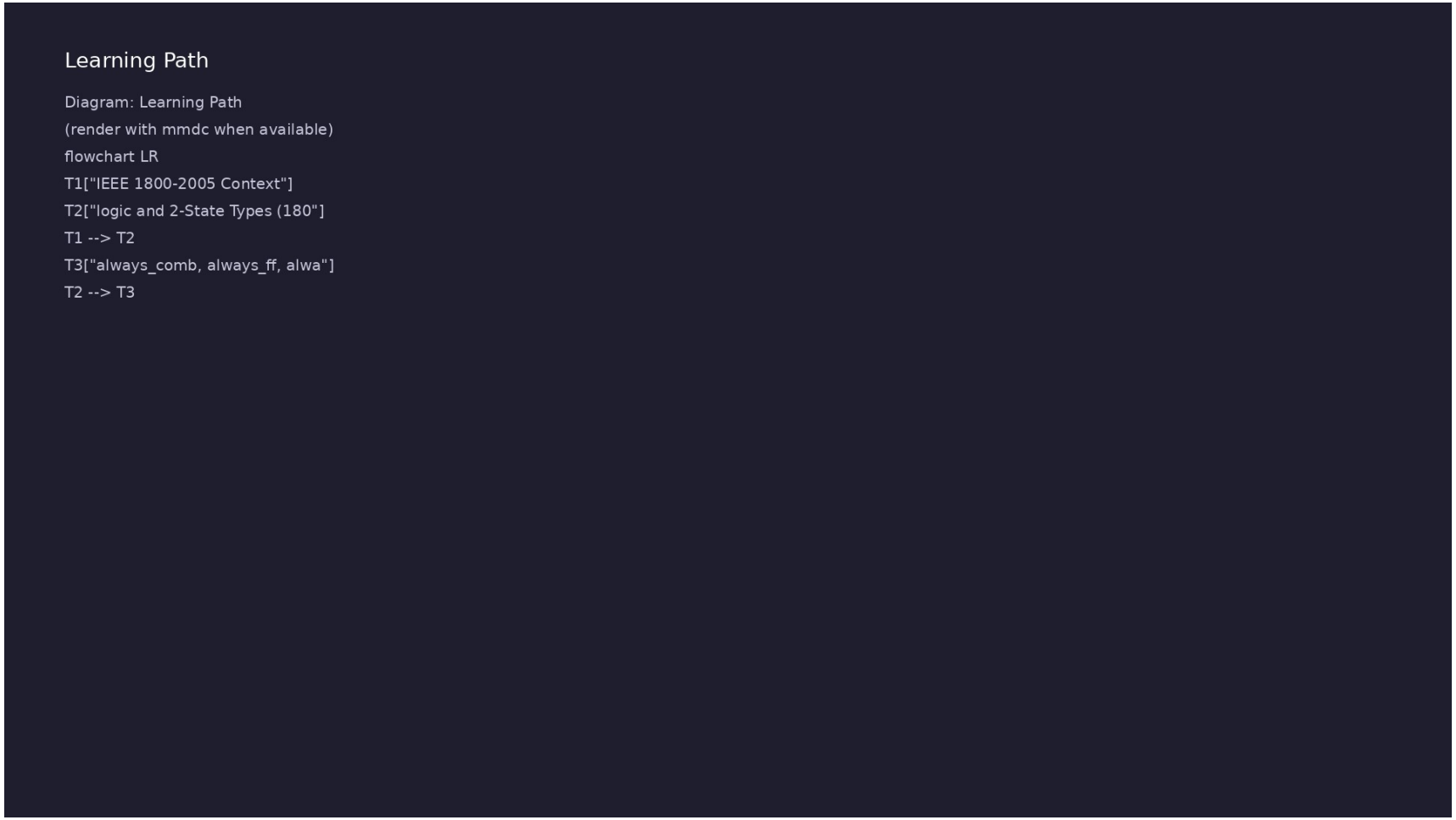
Learning objectives

- Master the first SystemVerilog standard (IEEE 1800-2005) for RTL design: logic types, always_comb/a…

Prerequisites

- See module README

Learning path



Master the first SystemVerilog standard (IEEE 1800-2005) for RTL design: logic types, always_comb/always_ff, interfaces, packages, and...

Overview

- This module covers IEEE Std 1800-2005, the first SystemVerilog standard. It extends Verilog (IEEE 1…

How to learn this module

Suggested learning path (1/2)

- Skim this document — Goal, Overview, and Topics Covered set the IEEE scope for Module 4.
- Study design architecture — See how DUTs, examples, and testbenches fit together under `module4/`.
- Work through labs in order — Open `module4/EXAMPLES.md` and run each `make clean && make run` from…
- Run the full module script — `./scripts/module4.sh` from the repo root to simulate all examples and…
- Complete the exercises — Try each exercise in this document before reading solutions or peeking at…

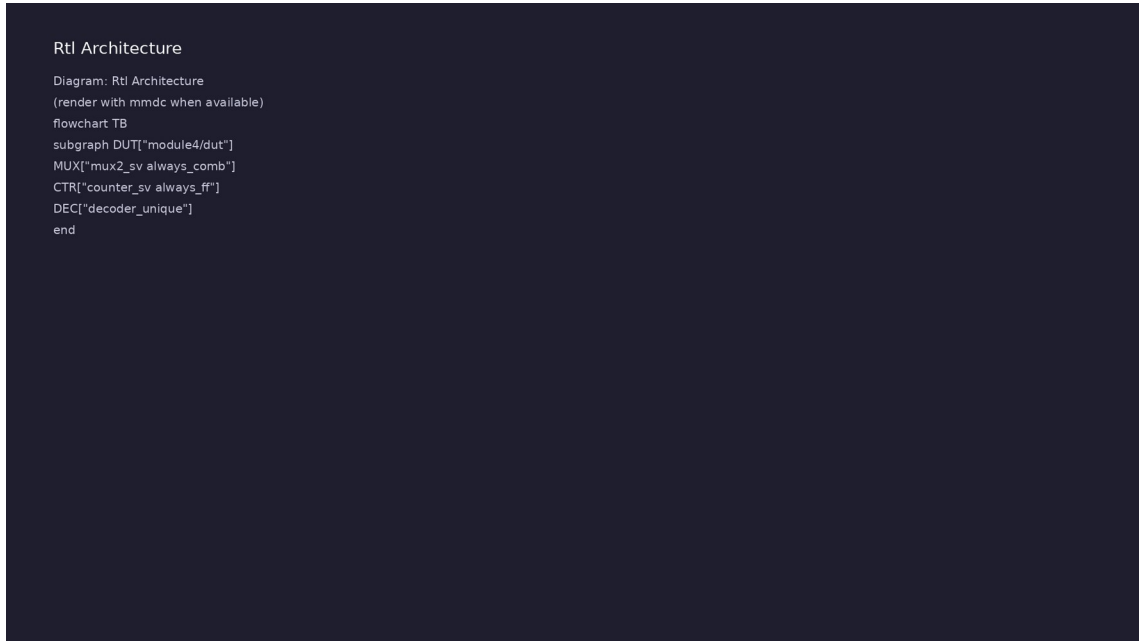
Suggested learning path (2/2)

- Review Common Pitfalls — Can you explain each mistake and the fix?
- Self-check — Use `module4/CHECKLIST.md` before starting the next module.

Design architecture

1. Repository layout (module4/) (1/2)

- First SystemVerilog module — .sv sources and design subset focus
- logic replaces wire/reg for single-driver RTL in most examples
- always_comb / always_ff / always_latch — explicit procedural intent
- interfaces and packages — connectivity and shared types



1. Repository layout (module4/) (2/2)

- scripts/module4.sh — requires simulator with SV design subset

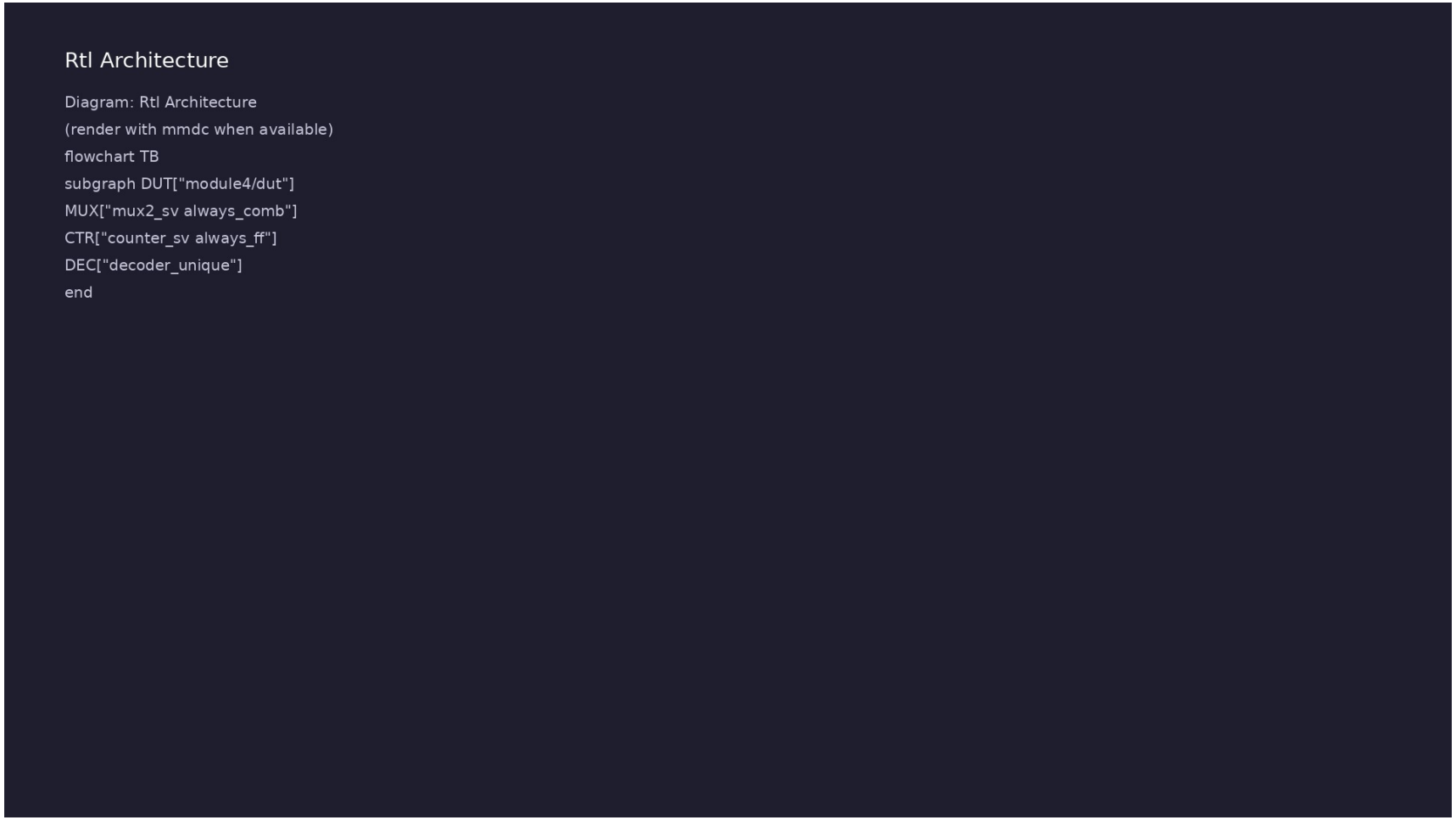
2. SystemVerilog design layering

- Packages hold typedefs, constants, functions — import in modules
- Interfaces bundle signals; modports define direction per connection
- always_comb infers sensitivity; always_ff targets flip-flops
- unique/priority case — standard semantics for full case checking

3. File and compilation units

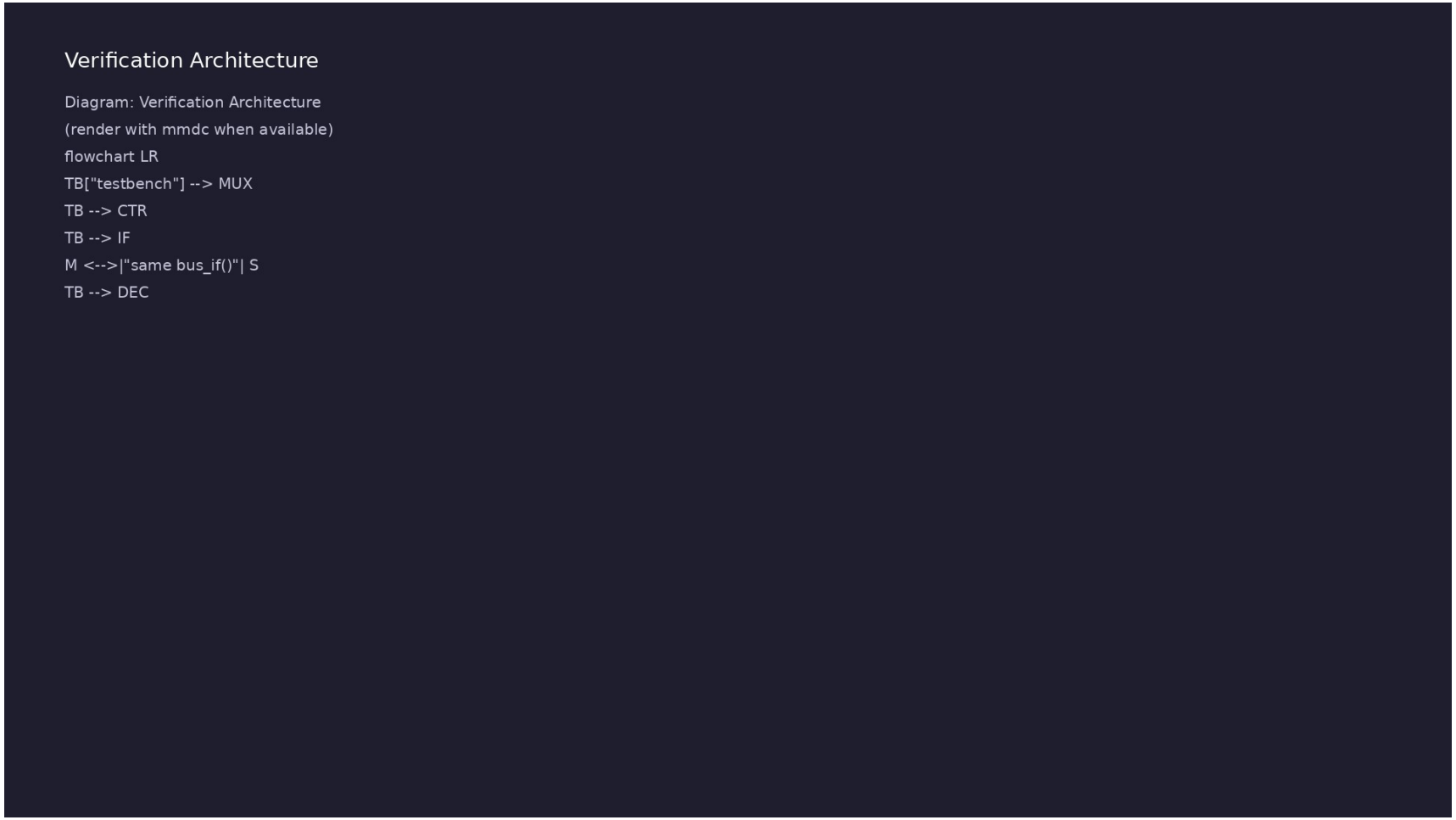
- .sv extension signals
  SystemVerilog to the tool
- Package before import; interface
  before modport connection
- See
  verification_architecture.png —
  TB still drives DUT via ports or
  interface

RTL block diagram (reference)



Module 4: DUT hierarchy and signal flow.

Verification / testbench diagram (reference)



Module 4: stimulus, observation, and checking.

logic ports and always_comb

```
/*
 * always_comb and always_ff - IEEE 1800-2005
 * Combinational: always_comb (inferred sensitivity). Sequential:
always_ff (one clock).
 */

module always_comb_ff_demo (
    input  logic       clk,
    input  logic       rst_n,
    input  logic       a,
    input  logic       b,
    input  logic       c,
    input  logic       d,
    output logic       y_comb,
    output logic       z_comb,
    output logic       q
);
```

Interface + modport connection

```
/*
 * Interface and modports - IEEE 1800-2005
 * One bundle; modports define direction per role.
 */

interface simple_bus_if;
    logic [7:0] data;
    logic       valid;
    logic       ready;
    logic       clk;
    logic       rst_n;

    modport master (output data, valid, input ready, clk, rst_n);
    modport slave  (input data, valid, output ready, input clk, rst_n);
endinterface
```

```
# ...
```

Execution & simulation flow

How the example runs (toolchain)

- Makefile: Verilator + UVM_HOME compile RTL, interface, and test_*.sv
- UVM phases: build → connect → run_test → report (same pattern as Module 2)
- Sequence → driver → DUT → monitor → scoreboard (read SCOREBOARD at end)
- Typical command: make SIM=verilator TEST=test_* (see module4 demo slide)
- Loopback / hooks connect monitor observations to scoreboard checks

Key files to study

Open these in the repo

- module4/examples/logic_ports/logic_ports.sv
- module4/examples/procedural/always_comb_ff.sv
- module4/examples/interfaces/bus_if_example.sv
- module4/examples/packages/pkg_util.sv
- scripts/module4.sh

Verification & testing methods

1. Running SystemVerilog labs

- cd module4/examples/<name> && make clean && make run
- ./scripts/module4.sh — full module; some labs need full SV support
- iverilog supports subset; check build.log if a feature is unsupported

2. What to check in simulation

- logic ports compile without separate wire/reg declarations
- always_comb blocks reject clock/reset inside (tool error = correct)
- Interface examples connect via modport, not dozens of scalar ports

Package import pattern

```
/*
 * Package and import - IEEE 1800-2005
 * Shared types, parameters, functions.
 */

package pkg_util;
    parameter int WIDTH = 8;
    typedef logic [WIDTH-1:0] word_t;

    function automatic logic [7:0] parity(input logic [31:0] v);
        parity = ^v;
    endfunction
endpackage

module alu (
    input  logic [31:0] a,
# ...
```

Syllabus topics

Protocol & design details

Detail: What 1800-2005 Adds Over 1364-2005 (Design Subset) (1)

- logic: Single type for single-driver nets/variables (replaces wire/reg in most RTL)
- 2-state types: `bit`, `int`, `byte`, etc. (mainly for testbenches; some RTL use)
- always_comb, always_ff, always_latch: Explicit procedural blocks with tool checks
- Interfaces and modports: Bundle signals and define direction per connection

Detail: What 1800-2005 Adds Over 1364-2005 (Design Subset) (2)

- Packages and import: Shared types, constants, and functions across modules
- unique case, priority case: Standard semantics for case statements (no tool-specific attributes)
- Operators: `inside`, wildcard `==?`, and other enhancements
- Verification: Classes, assertions (SVA), coverage, etc. (covered in later modules or other courses)

Detail: What This Module Emphasizes

- Design/RTL: logic, always_comb/always_ff, interfaces, packages, unique/priority case
- Not in depth: Classes, SVA, coverage, constrained random (those are verification-focused)

Detail: Relation to 1364-2005

- 1364-2005 (Verilog) is a subset of 1800-2005 (SystemVerilog).
- Valid 1364-2005 RTL is valid 1800-2005 RTL; you can migrate incrementally (e.g. wire→logic, always…

Detail: logic (4-State, Single Driver) (1)

- logic: Replaces `wire` and `reg` when there is a single driver; can be used in both continuous and…
- Rule: A `logic` variable must have exactly one driver (one assign, or one always block, or one port…
- 4-state: 0, 1, X, Z (same as wire/reg).
- Ports: `input logic`, `output logic`; no need to choose wire vs reg for single-driver ports.

Detail: logic (4-State, Single Driver) (2)

- Internal: Use `logic` for single-driver signals; use `wire` only when needed (e.g. multiple drivers...

Detail: bit and 2-State Types (1800)

- bit: 2-state (0, 1); no X or Z. Often used in testbenches and for indices.
- int, byte, shortint, longint: Integer types; 2-state unless declared with a 4-state base.
- Use in RTL: Optional (e.g. loop indices, parameters); many RTL designs use `logic` and `integer`/`r...

Detail: always_comb

- Intent: Combinational logic; no storage; outputs are a function of inputs only.
- Sensitivity: Automatically inferred (like `always @*`); must not contain latches.
- Tool check: Simulator/synthesis can warn if the block implies storage or is sensitive to clock edge…
- Prefer over `always @*` when the tool supports it: clearer intent and better checking.

Detail: always_ff

- Intent: Sequential logic (flip-flops); use nonblocking assignment (`<=`) only.
- Sensitivity: Must list exactly one clock (and optionally async reset); no other event controls.
- Tool check: Can warn if blocking assignment or incomplete sensitivity is used.

Detail: always_latch

- Intent: Explicit latch (level-sensitive storage).
  Use sparingly; usually latches are unintended.
- When: When a latch is truly intended (e.g.
  transparent latch); otherwise use always_comb or
  always_...
- Recommendation: Prefer always_comb or always_ff; use
  always_latch only when a latch is desired.

Detail: Interface Declaration

- Signals: Declared once inside the interface; shared by all modules that use it.
- modport: Defines a view (direction) for a given role (e.g. master sees data as output, slave sees d…

Detail: Using an Interface in a Module

- Port: Single interface port with a modport (e.g. `bus_if.master bus`).
- Access: `bus.signal_name`; direction is enforced by the modport.

Detail: Top-Level Connection

- One interface instance: Connects master and slave;
  no long port lists.

Detail: Package Declaration

- Contents: parameter, localparam, typedef, function, task, etc. (no always blocks).
- Scope: Items are in namespace `pkg_util::` until imported.

Detail: Import and Use

- import pkg_util::*: Import all names from the package (can cause name clashes).
- import pkg_util::item: Import specific items; safer when many packages are used.

Detail: unique case

- Semantics: At most one case item must match at runtime; if none or more than one matches, a runtime…
- Use: When the design guarantees at most one match (e.g. one-hot encoding, mutually exclusive condit…
- Synthesis: Can be implemented as parallel priority logic; no inferred priority order.

Detail: priority case

- Semantics: The first matching item has priority; if no item matches, a runtime error can be reporte…
- Use: When priority order is intended (e.g. interrupt mask).
- Synthesis: Can be implemented as a priority chain (if-then-else or similar).

Verification flow (reference)

Stimulus, DUT response, and check points for this module.

1. IEEE 1800-2005 Context (1/2)

- logic: Single type for single-driver nets/variables (replaces wire/reg in most RTL)
- 2-state types: `bit`, `int`, `byte`, etc. (mainly for testbenches; some RTL use)
- always_comb, always_ff, always_latch: Explicit procedural blocks with tool checks
- Interfaces and modports: Bundle signals and define direction per connection
- Packages and import: Shared types, constants, and functions across modules
- unique case, priority case: Standard semantics for case statements (no tool-specific attributes)

1. IEEE 1800-2005 Context (2/2)

- Operators: `inside`, wildcard `==?`, and other enhancements
- Verification: Classes, assertions (SVA), coverage, etc. (covered in later modules or other courses)
- Design/RTL: logic, always_comb/always_ff, interfaces, packages, unique/priority case
- Not in depth: Classes, SVA, coverage, constrained random (those are verification-focused)
- 1364-2005 (Verilog) is a subset of 1800-2005 (SystemVerilog).
- Valid 1364-2005 RTL is valid 1800-2005 RTL; you can migrate incrementally (e.g. wire→logic, always...

2. logic and 2-State Types (1800) (1/2)

- logic: Replaces `wire` and `reg` when there is a single driver; can be used in both continuous and...
- Rule: A `logic` variable must have exactly one driver (one assign, or one always block, or one port...
- 4-state: 0, 1, X, Z (same as wire/reg).
- Ports: `input logic`, `output logic`; no need to choose wire vs reg for single-driver ports.
- Internal: Use `logic` for single-driver signals; use `wire` only when needed (e.g. multiple drivers...
- bit: 2-state (0, 1); no X or Z. Often used in testbenches and for indices.

2. logic and 2-State Types (1800) (2/2)

- int, byte, shortint, longint: Integer types; 2-state unless declared with a 4-state base.
- Use in RTL: Optional (e.g. loop indices, parameters); many RTL designs use `logic` and `integer`/`r...

3. always_comb, always_ff, always_latch (1800) (1/2)

- Intent: Combinational logic; no storage; outputs are a function of inputs only.
- Sensitivity: Automatically inferred (like `always @*`); must not contain latches.
- Tool check: Simulator/synthesis can warn if the block implies storage or is sensitive to clock edge…
- Prefer over `always @*` when the tool supports it: clearer intent and better checking.
- Intent: Sequential logic (flip-flops); use nonblocking assignment (`<=`) only.
- Sensitivity: Must list exactly one clock (and optionally async reset); no other event controls.

3. always_comb, always_ff, always_latch (1800) (2/2)

- Tool check: Can warn if blocking assignment or incomplete sensitivity is used.
- Intent: Explicit latch (level-sensitive storage). Use sparingly; usually latches are unintended.
- When: When a latch is truly intended (e.g. transparent latch); otherwise use always_comb or always_...
- Recommendation: Prefer always_comb or always_ff; use always_latch only when a latch is desired.

4. Interfaces and Modports (1800)

- Signals: Declared once inside the interface; shared by all modules that use it.
- modport: Defines a view (direction) for a given role (e.g. master sees data as output, slave sees d…
- Port: Single interface port with a modport (e.g. `bus_if.master bus`).
- Access: `bus.signal_name`; direction is enforced by the modport.
- One interface instance: Connects master and slave; no long port lists.

5. Packages and import (1800)

- Contents: parameter, localparam, typedef, function, task, etc. (no always blocks).
- Scope: Items are in namespace `pkg_util::` until imported.
- import pkg_util::*: Import all names from the package (can cause name clashes).
- import pkg_util::item: Import specific items; safer when many packages are used.

6. unique case and priority case (1800)

- Semantics: At most one case item must match at runtime; if none or more than one matches, a runtime…
- Use: When the design guarantees at most one match (e.g. one-hot encoding, mutually exclusive condit…
- Synthesis: Can be implemented as parallel priority logic; no inferred priority order.
- Semantics: The first matching item has priority; if no item matches, a runtime error can be reporte…
- Use: When priority order is intended (e.g. interrupt mask).
- Synthesis: Can be implemented as a priority chain (if-then-else or similar).

7. Comparison: 1364-2005 vs 1800-2005 (Design)

- See docs/MODULE4.md

Hands-on examples

Module 4 toolchain check

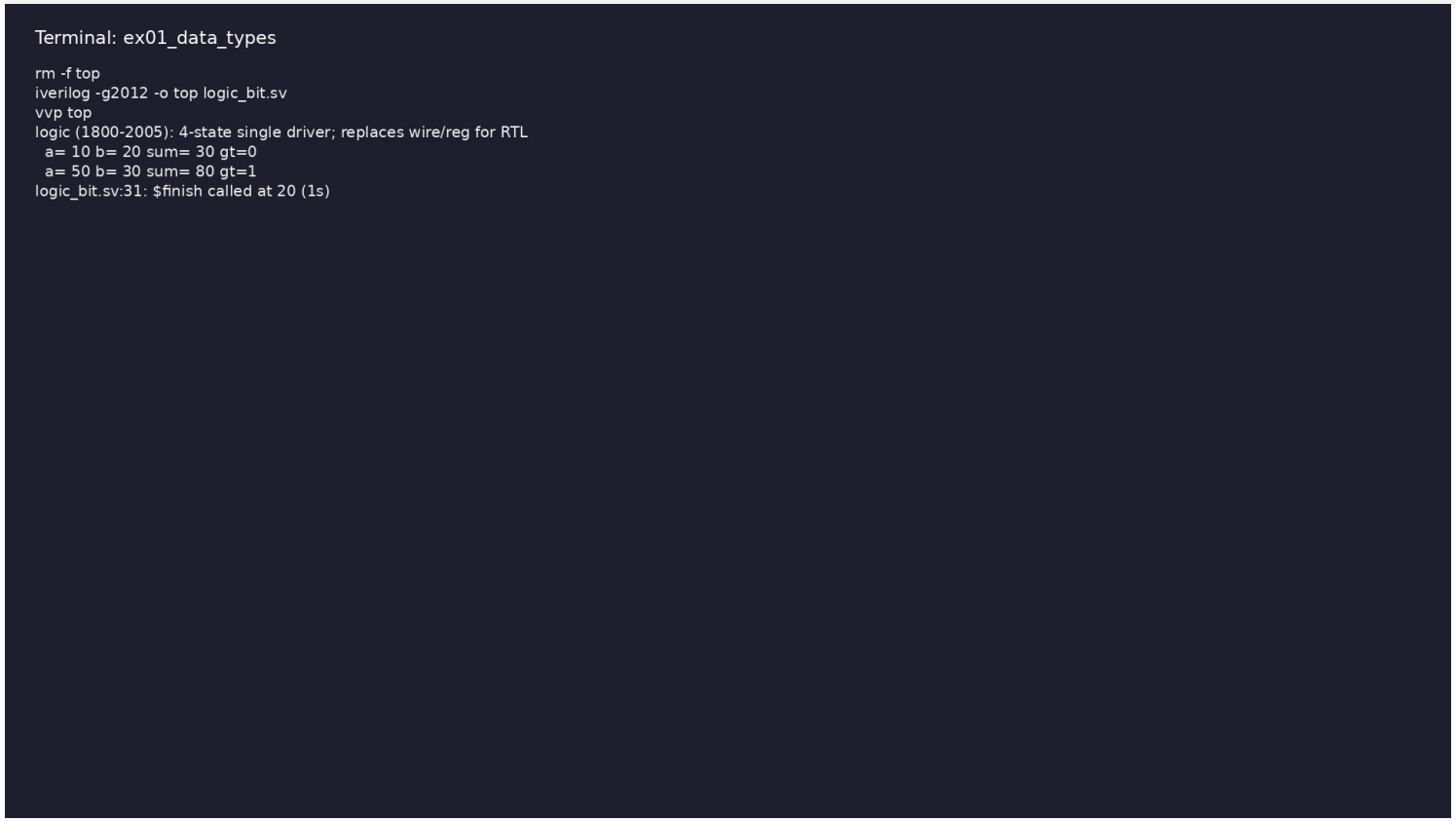
```
$ command -v iverilog >/dev/null && echo "Toolchain ready: $(iverilog
      -V 2>&1 | head -1)"
```

Example 1: logic for ports and internal single-driver signals

- Single driver for logic; 4-state

Demo: logic for ports and internal single-driver signals

```
$ cd module4/examples/data_types && make clean && make run
```

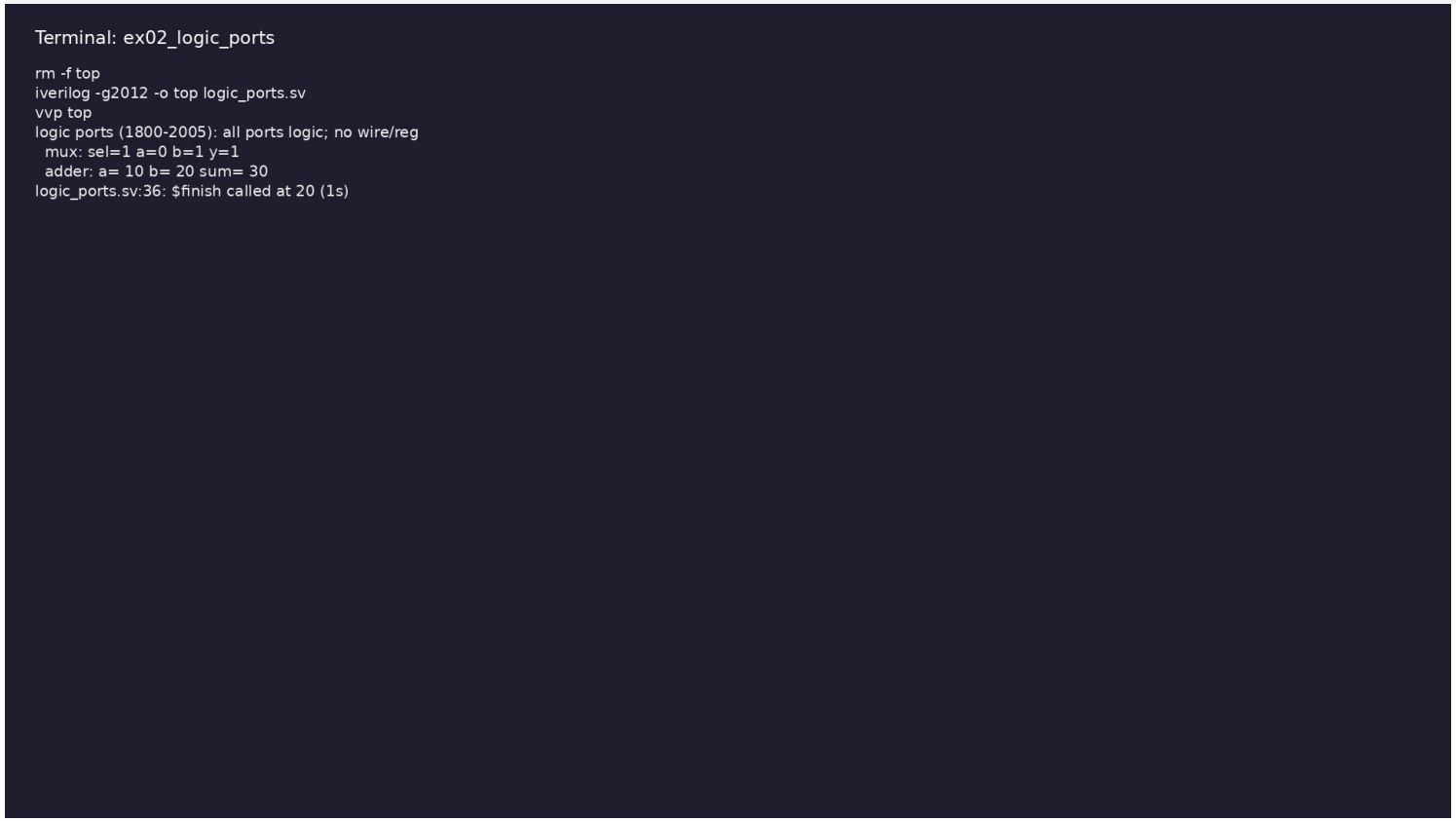


Example 2: All ports logic (mux, adder); no wire/reg choice

- assign and always_comb can drive output logic

Demo: All ports logic (mux, adder); no wire/reg choice

```
$ cd module4/examples/logic_ports && make clean && make run
```

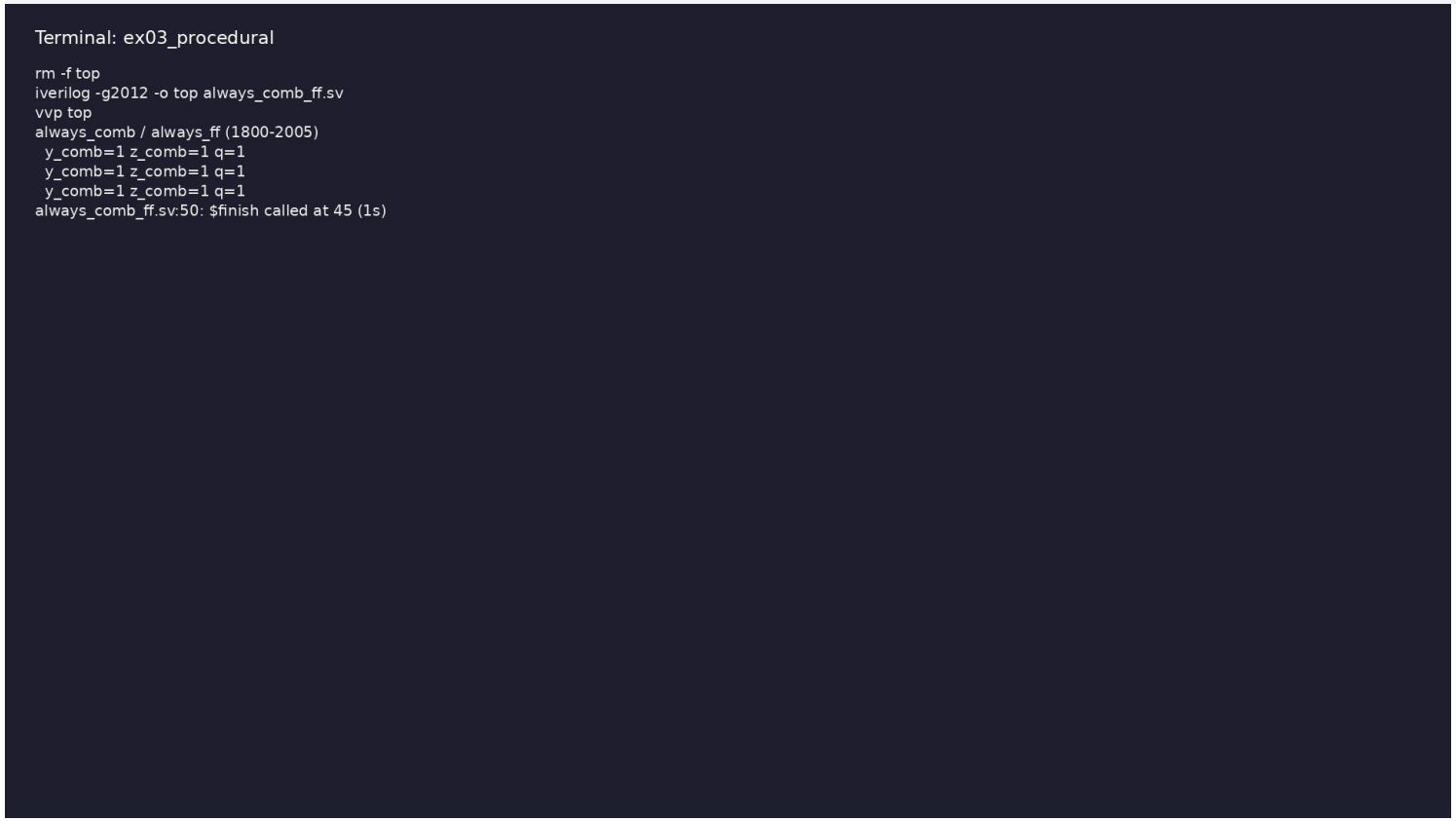


Example 3: Combinational with always_comb; sequential with always_ff

- Explicit intent; tool checks; no latch in always_comb

Demo: Combinational with always_comb; sequential with always_ff

```
$ cd module4/examples/procedural && make clean && make run
```



Example 4: Explicit latch when intended (e.g. transparent latch)

- Use when latch is desired; prefer always_comb/always_ff otherwise

Demo: Explicit latch when intended (e.g. transparent latch)

```
$ cd module4/examples/always_latch && make clean && make run
```

Example 5: Single assign to one output; single always_ff to another

- logic must have exactly one driver

Demo: Single assign to one output; single always_ff to another

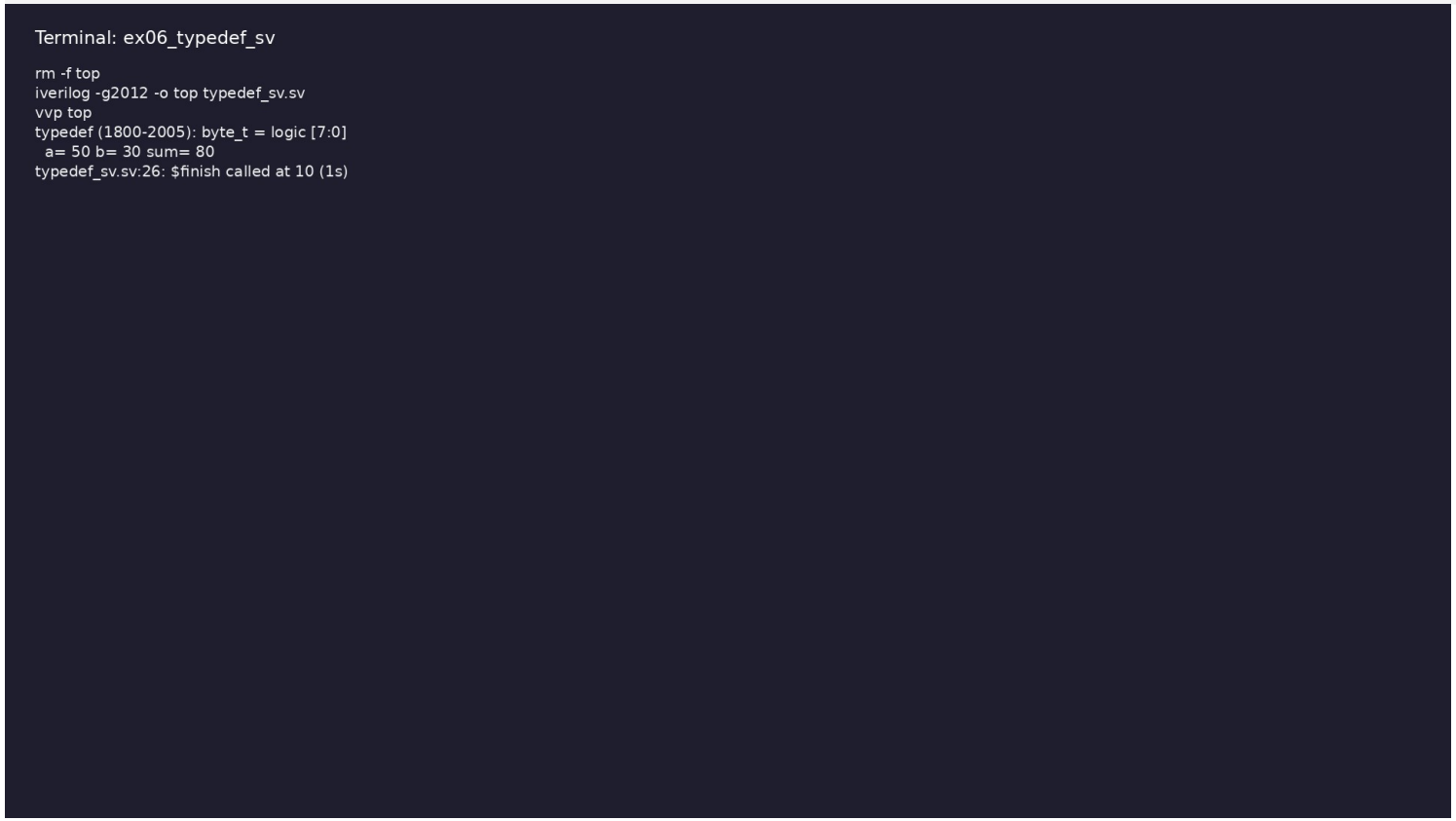
```
$ cd module4/examples/one_driver_logic && make clean && make run
```

Example 6: typedef in module (e.g. byte_t = logic [7:0])

- User-defined types for clarity

Demo: typedef in module (e.g. byte_t = logic [7:0])

```
$ cd module4/examples/typedef_sv && make clean && make run
```

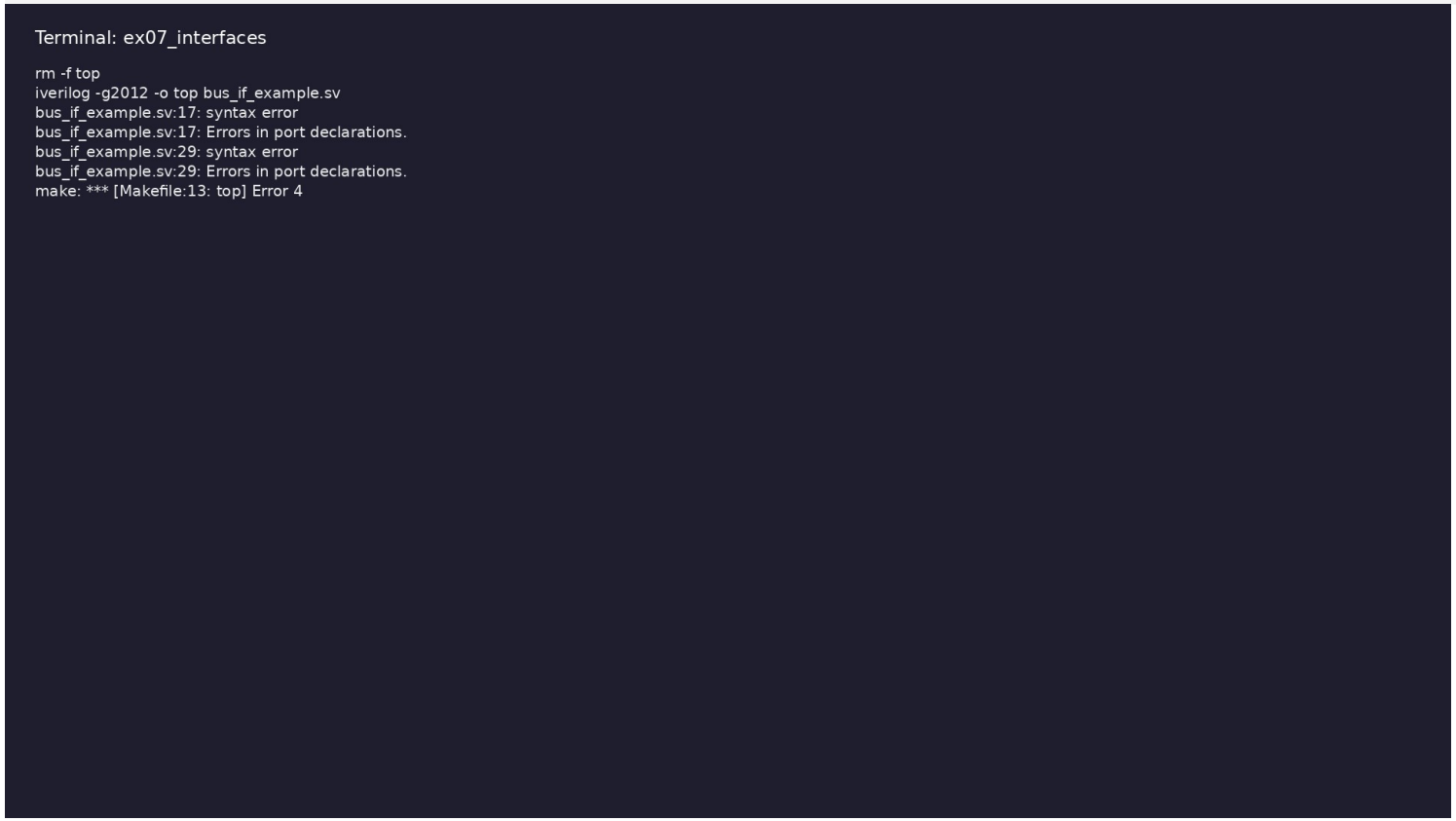


Example 7: Interface definition, modports, connection in top

- One bundle per connection; direction per modport

Demo: Interface definition, modports, connection in top

```
$ cd module4/examples/interfaces && make clean && make run
```



Example 8: Package with types/params/functions; import in modules

- Namespace; wildcard vs specific import

Demo: Package with types/params/functions; import in modules

```
$ cd module4/examples/packages && make clean && make run
```

Example 9: Package with typedef and min_word function; import in ALU

- Shared types and helpers across modules

Demo: Package with typedef and min_word function; import in ALU

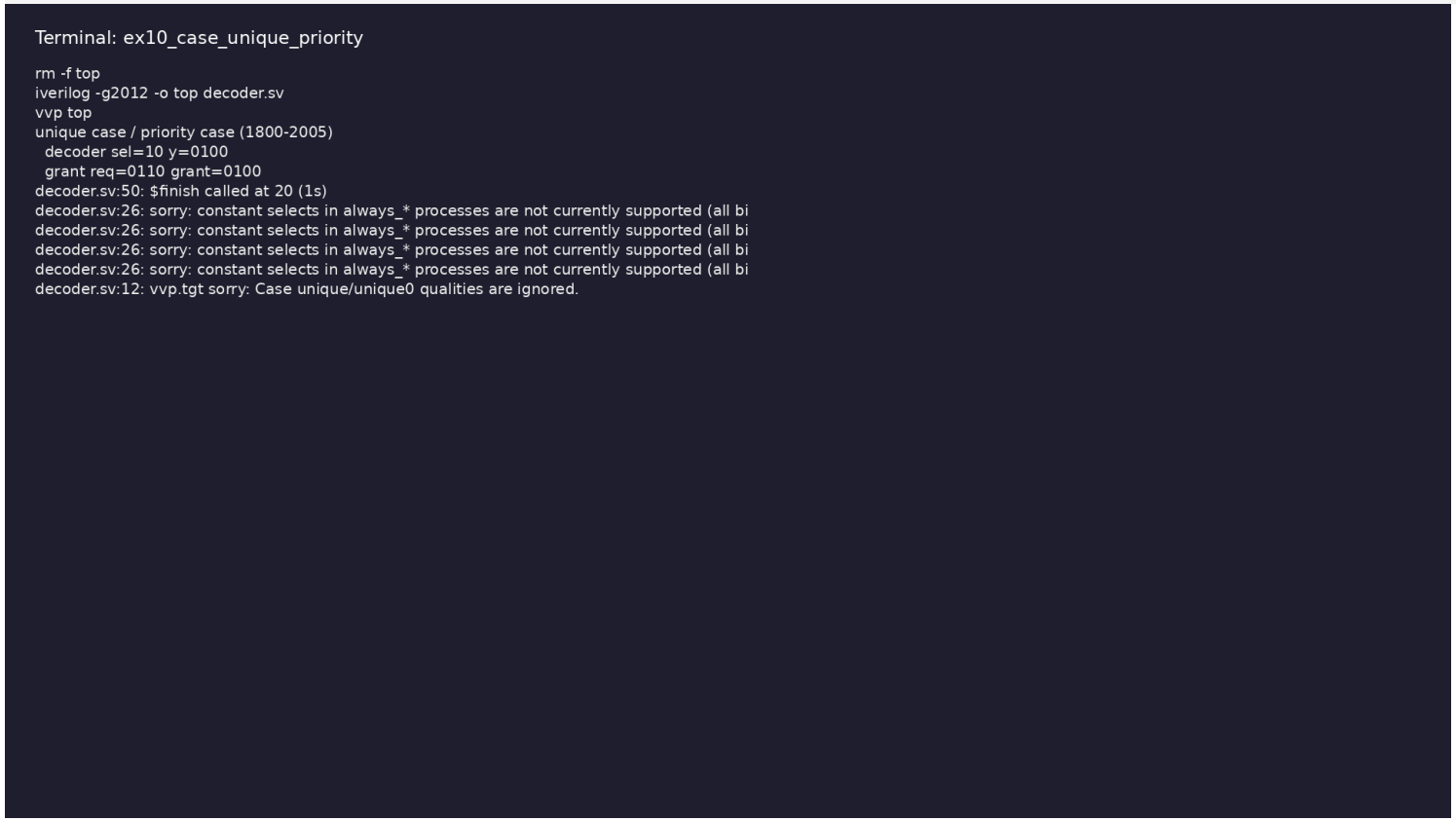
```
$ cd module4/examples/package_typedef && make clean && make run
```

Example 10: unique case (at most one match); priority case (first mat…

- Standard semantics; synthesis and simulation behavior

Demo: unique case (at most one match); priority case (first match)

```
$ cd module4/examples/case_unique_priority && make clean && make run
```

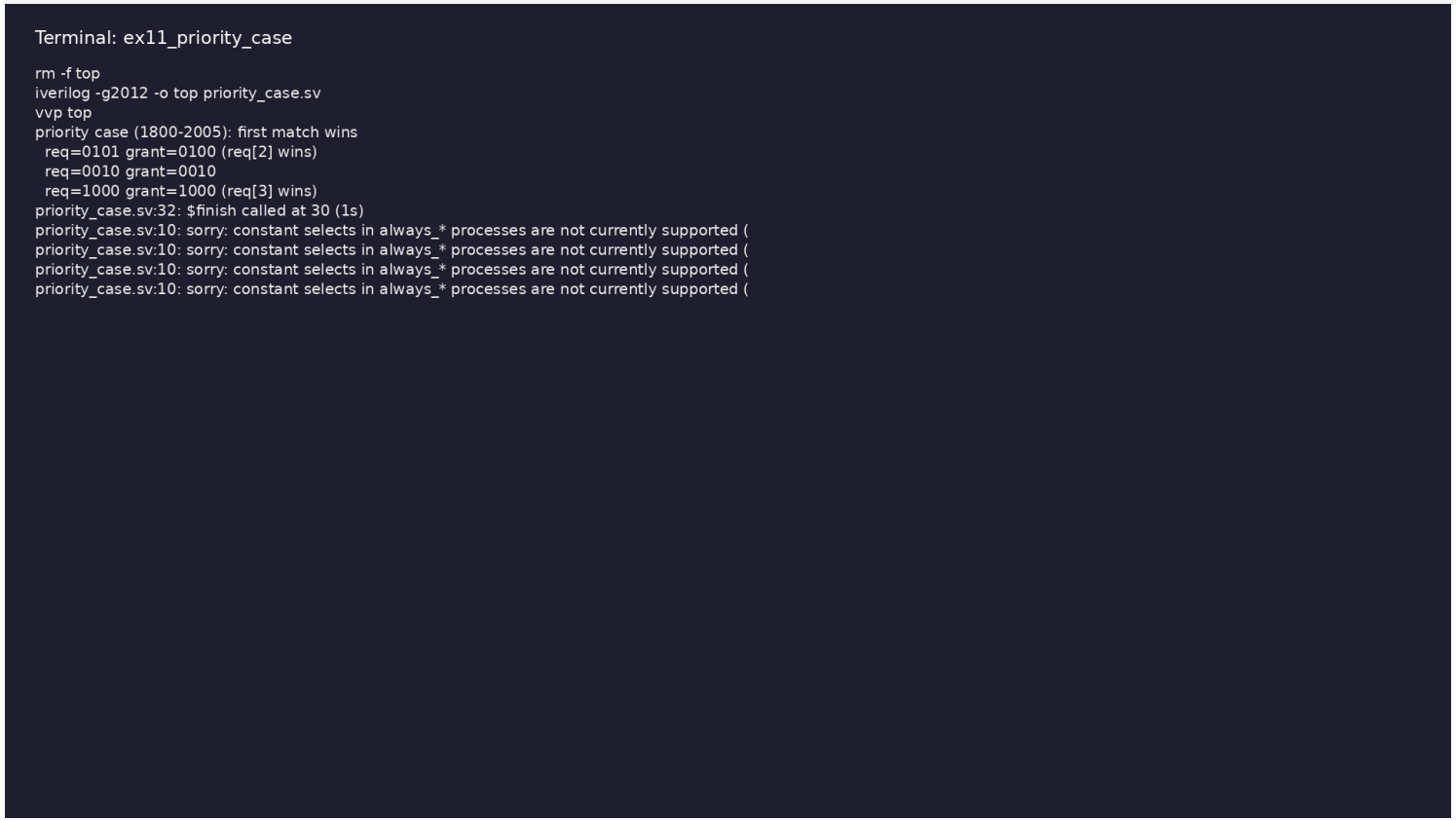


Example 11: Arbiter with priority case (first req wins)

- priority case for arbiter, interrupt mask

Demo: Arbiter with priority case (first req wins)

```
$ cd module4/examples/priority_case && make clean && make run
```



Example 12: Same small design in 1364-2005 style vs 1800-2005 style

- wire/reg→logic, always @*→always_comb, always @(posedge clk)→always_ff

Demo: Same small design in 1364-2005 style vs 1800-2005 style

```
$ cd module4/examples/migration && make clean && make run
```

Common pitfalls

Multiple Drivers on logic

- Mistake: Driving the same logic from two assign statements or two always blocks.
- Reality: logic allows only one driver; multiple drivers are illegal or undefined.
- Correct: Use one assign or one always per logic; for multiple drivers use wire (e.g. tri-state).
- Why: logic is for single-driver RTL; wire is for resolved (multi-driver) nets.

Using always_comb with Latched Behavior

- Mistake: Incomplete branches in always_comb (e.g. no default in case) and expecting no latch.
- Reality: Tools may infer a latch or report a warning.
- Correct: Assign every output in every path, or provide default before case.
- Why: always_comb is intended for pure combinational logic; latches require always_latch if desired.

Wrong Modport or Direction

- Mistake: Driving an input in a modport or reading an output that is not in the modport.
- Reality: Compiler or elaboration error; or wrong connectivity.
- Correct: Use the modport that matches the module role (master vs slave); check direction in the int…
- Why: Modports enforce direction; using the wrong view breaks the contract.

Forgetting to import Package

- Mistake: Using a package type or function without importing.
- Reality: Identifier not found; compile error.
- Correct: Add `import pkg_name::*;` or `import pkg_name::item;` before use.
- Why: Package names are in a separate namespace until imported.

unique case with Multiple Matches

- Mistake: Writing unique case when two items can match (e.g. overlapping or X in select).
- Reality: Runtime error or undefined behavior.
- Correct: Use unique case only when at most one item can match; use default for catch-all; or use pr…
- Why: unique case has well-defined semantics; violating them breaks verification and synthesis.

Practice & assessment

What you should know (1/2)

- ✓ Use logic for single-driver nets and variables in RTL
- ✓ Use always_comb for combinational logic and always_ff for sequential logic
- ✓ Define and use interfaces with modports for bundled connectivity
- ✓ Define packages and import them in modules
- ✓ Use unique case and priority case with correct semantics
- ✓ Compare 1364-2005 and 1800-2005 design constructs and migrate simple RTL from Verilog to SystemVe…

What you should know (2/2)

- ✓ Decide when to adopt 1800 design features vs stay with 1364

Exercises

- logic and always_comb/always_ff
- Interface
- Package
- unique case
- Migration Summary

Exercises

- Multiple Drivers on logic
- Using always_comb with Latched Behavior
- Wrong Modport or Direction
- Forgetting to import Package
- unique case with Multiple Matches

Summary & next steps

- Master the first SystemVerilog standard (IEEE 1800-2005) for RTL design: logic types, always_comb/a…
- Complete module4/CHECKLIST.md
- Review module4/EXAMPLES.md and run each lab
- Next: Next module in course